MNIST Digit Classification Model Deployment on Google Cloud

- By Saloni Tripathi

Executive Summary

This project focused on deploying a handwritten digit recognition model as a secure FAST API on Google Cloud Platform. In this project, I trained a neural network on the MNIST dataset (images of digits 0–9) using TensorFlow. The trained model is containerized (Docker) and hosted on Cloud Run and exposed via an API Gateway to provide a stable HTTPS endpoint. Access is secured by requiring an API key. The deployment was successful, and the model runs with high accuracy.

Problem and Dataset

The task is a multi-class classification problem where the model needs to recognize handwritten digits (0–9). I used the **MNIST** dataset, which is frequently used for digit recognition. MNIST consists of 60,000 training and 10,000 test grayscale images, each 28×28 pixels, labeled with a digit from 0–9. This dataset is accessible via TensorFlow/Keras (`tf.keras.datasets.mnist`). Our goal is to train a model that, given an input image, predicts which of the 10-digit classes it represents.

Model Development

- **Preprocessing:** Pixel values (0–255) were scaled to the range [0,1] by dividing by 255.0. This normalization speeds up training and helps convergence. The labels (0–9) were converted to one-hot vectors (length 10) since this is a 10-way classification problem.
- **Network Architecture:** I defined a simple feed-forward neural network using Keras's Sequential API. The first layer is Flatten(`input_shape=(28,28)`) to convert each 28×28 image into a 784-length vector. This is followed by a fully-connected Dense layer with 128 neurons and ReLU activation, and a final Dense(10) layer with softmax activation to output class probabilities.
- **Training:** The model was compiled with the Adam optimizer and categorical cross-entropy loss. It was trained for 5 epochs with a batch size of 32. During the training process, accuracy and loss were logged. As expected for MNIST, the model quickly converged, giving ~98% training accuracy after 5 epochs and ~97.8% accuracy on the test set. I saved the trained model in .h5 format and printed out accuracy/loss before and after saving to verify consistency.

Deployment Architecture Diagram

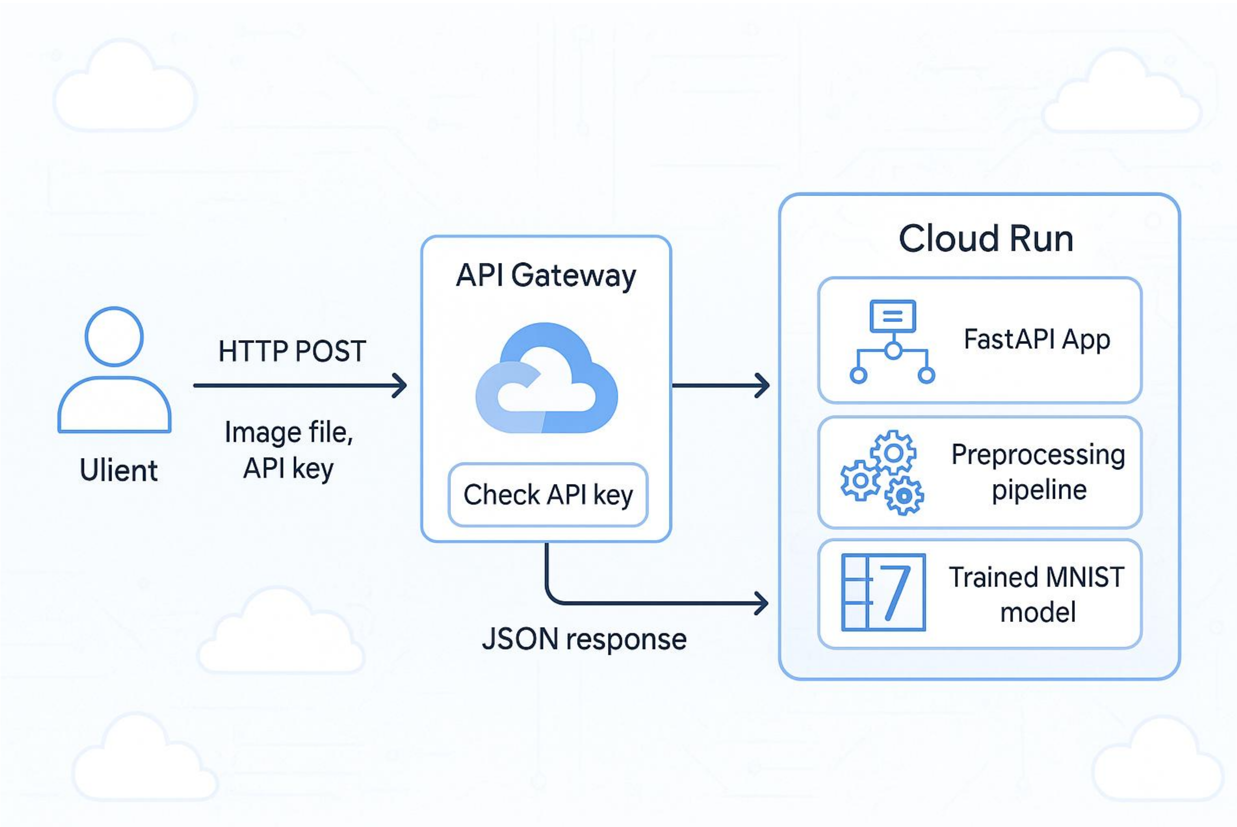


Fig 1. Deployment Architecture

Deployment Process

- **Containerization:** I implemented the prediction logic in a Python FastAPI, app that loads the saved TensorFlow model and handles image uploads and predictions. I also created a Dockerfile (based on Python 3.x) to copy the application code and install dependencies.
- **Pushing to Container Registry:** After building the Docker (`docker build --no-cache -t gcr.io/superb-ethos-462618-k3/mnist-api:latest`), I ran gcloud auth configure-docker to allow Docker to authenticate with Google Container Registry (GCR), then tagged and pushed the image to my project's registry (`docker push gcr.io/superb-ethos-462618-k3/mnist-api:latest`). This made the image available for Cloud Run.
- **Deploy to Cloud Run:** Using the GCP Console, I deployed the container image to Cloud Run. I specified 1 Gi of memory (upgraded from the default 512 MiB after testing) to ensure enough RAM for the model and increased the timeout to 30 seconds. After the image was successfully deployed, Cloud Run gave an HTTPS endpoint (URL) for the service.

- **API Gateway Configuration:** I wrote an OpenAPI (Swagger) YAML file defining the API, including a /predict-image path for POST requests. In the spec's x-google-backend section, I set the address to the Cloud Run service URL. Then I used '`gcloud api-gateway api-configs create mnist-config-v2 --api=mnist-api --openapi-spec=openapi.yaml --project=superb-ethos-462618-k3`' to create a new API config from the OpenAPI spec and to deploy an API Gateway for our service. The gateway provides a fixed hostname.

- **Securing with API Key:** I generated a new API key in the GCP Console and restricted it to the gateway API. Then I added a security definition (api_key) requiring the key as a query parameter in the openapi.yaml file. After deploying the gateway with this config, any request to the endpoint must include '`?key=<API_KEY>`'. For example:

  ```
  curl.exe -X POST -F "file=@digit.png" https://mnist-gateway-a1xxbu86.uc.gateway.dev/predict-image?key=AIzaSyAKrnknzBkcjyn5b0_Os9d73fC8AboTwoI"
  ```

  A successful request returns a JSON payload like {"prediction":7}. Requests missing the key will fail with a permission error.

Challenges Faced

During deployment, several challenges were encountered and systematically resolved. Initially, the OpenAPI specification did not have operationId fields for each endpoint, which caused deployment errors with API Gateway as they are required fields. These were fixed by adding unique operationId values. Another issue arose when the Cloud Run container exceeded the default 512 MiB memory limit during inference, causing startup failures. I solved this issue by increasing the allocated memory to 1 Gi but after that the request was getting timed out, so I had to increase the timeout time from 15 seconds to 30 seconds. Another error that I got was API Gateway returned 403 PERMISSION_DENIED errors in early tests. This was due to the required API service (`<API_ID>-<hash>.apigateway.<PROJECT>.cloud.goog`) not being enabled in the GCP project. Enabling it using gcloud services enable resolved the issue.

Conclusion

In conclusion, I successfully deployed the MNIST digit classifier as a scalable cloud service. The model achieved high accuracy during training, and the deployed API now returns correct digit predictions on test images. The endpoint is secured via API Gateway with key-based authentication and runs on Google's serverless Cloud Run platform. Future enhancements could include supporting batch predictions (multiple images per request), improve preprocessing (e.g. better image normalization or augmentation), or use a more sophisticated model (like a convolutional network) for even higher accuracy. Cloud Run's GPU support can also be leveraged for acceleration.

Screenshots

```
Epoch 1/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.8783 - loss: 0.4351 - val_accuracy: 0.9598 - val_loss: 0.1348
Epoch 2/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - accuracy: 0.9648 - loss: 0.1232 - val_accuracy: 0.9690 - val_loss: 0.0986
Epoch 3/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.9770 - loss: 0.0775 - val_accuracy: 0.9742 - val_loss: 0.0828
Epoch 4/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 11s 3ms/step - accuracy: 0.9832 - loss: 0.0567 - val_accuracy: 0.9746 - val_loss: 0.0821
Epoch 5/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - accuracy: 0.9869 - loss: 0.0453 - val_accuracy: 0.9753 - val_loss: 0.0801
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9700 - loss: 0.0949
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file form
Test accuracy before saving: 0.9753, loss: 0.0801
Model saved to 'mnist_model.h5'
WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile_metrics` will be empty
Model loaded from 'mnist_model.h5'
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.9700 - loss: 0.0949
Test accuracy after loading: 0.9753, loss: 0.0801
```

Fig 2. Model Training and Test Accuracy

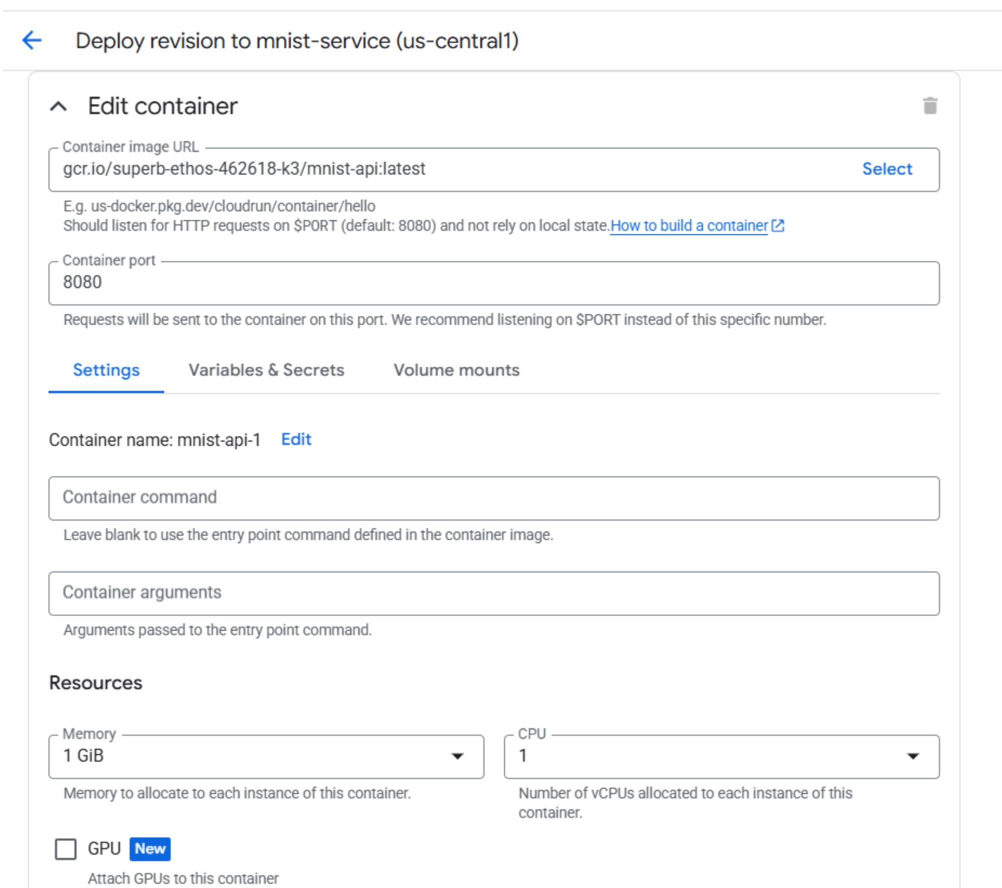


Fig 3. Google Cloud Run service

```
SEVERITY  TIME                      SUMMARY
  i       2025-06-22 14:56:34.649   POST  200  275 B  84 ms  /predict-image?key=AIzaSyAKrnknzBkcjyn5b0_Os9d73fC8AboTwoI
Explain this log entry   Copy   Expand nested fields   Hide log summary
```

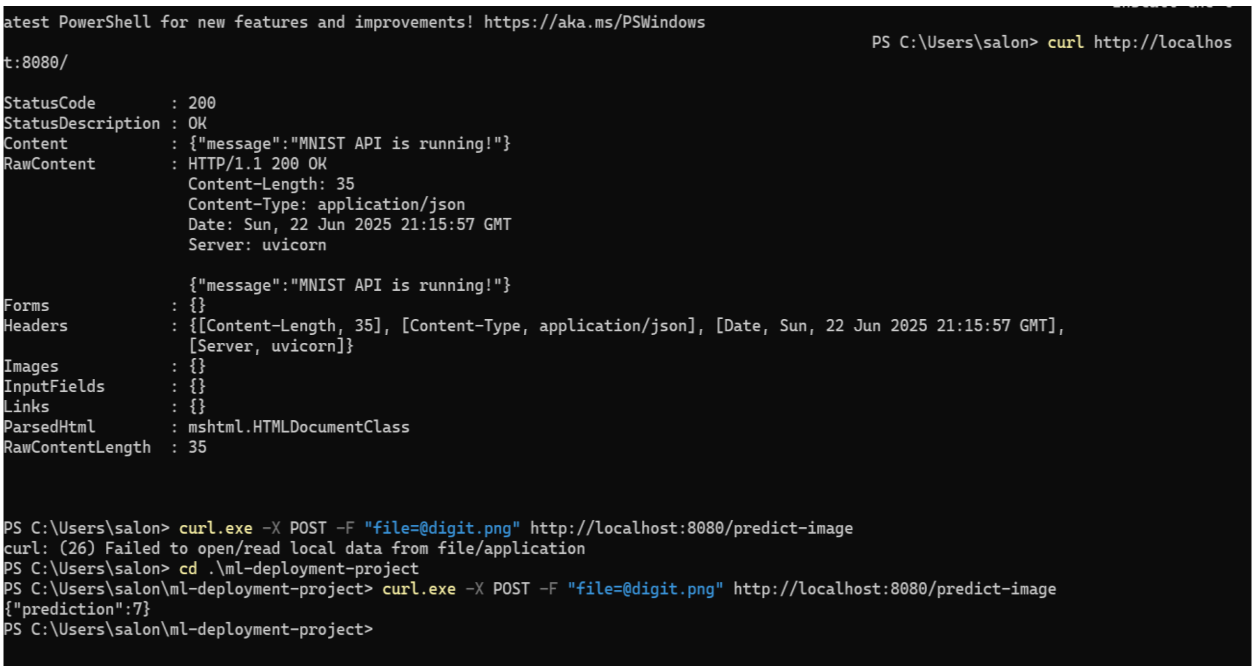

```
{
  httpRequest: {8}
  insertId: "9524a6d7-21c0-46b9-9815-d903a11a148548509089056396124390a1"
  jsonPayload: {
    api_key: "AIzaSyAKrnknzBkcjyn5b0_Os9d73fC8AboTwoI"
    api_key_state: "VERIFIED"
    api_method: "1.superb_ethos_462618_k3_endpoints_superb_ethos_462618_k3_cloud_goog.PredictImage"
    api_name: "1.superb_ethos_462618_k3_endpoints_superb_ethos_462618_k3_cloud_goog"
    api_version: "1.0.0"
    http_status_code: 200
    location: "us-central1"
    log_message: "1.superb_ethos_462618_k3_endpoints_superb_ethos_462618_k3_cloud_goog.PredictImage is called"
    producer_project_id: "superb-ethos-462618-k3"
    response_code_detail: "via_upstream"
    service_agent: "ESPv2/2.52.0"
    service_config_id: "mnist-config-v2-2eqdaeilos1k0"
    timestamp: 1750618594.6498973
  }
  logName: "projects/superb-ethos-462618-k3/logs/mnist-api-2ysvxq2cru91q.apigateway.superb-ethos-462618-k3.cloud.goog%2Fendpoints_log"
  receiveTimestamp: "2025-06-22T18:56:36.369723743Z"
  resource: {2}
```


Fig 4. API Gateway Logs

```
PS C:\Users\salon> cd .\ml-deployment-project
PS C:\Users\salon\ml-deployment-project> docker run --rm -p 8080:8080 mnist-api-local
Error: Option '--port' requires an argument.
PS C:\Users\salon\ml-deployment-project> docker run --rm -p 8080:8080 -e PORT=8080 mnist-api-local
2025-06-22 21:15:38.261465: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numer
ical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable
 `TF_ENABLE_ONEDNN_OPTS=0`.
2025-06-22 21:15:38.319127: I external/local_xla/xla/tsl/cuda/cudart_stub.cc:32] Could not find cuda drivers on your machine, GPU wil
l not be used.
2025-06-22 21:15:38.754260: I external/local_xla/xla/tsl/cuda/cudart_stub.cc:32] Could not find cuda drivers on your machine, GPU wil
l not be used.
2025-06-22 21:15:39.086383: E external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:467] Unable to register cuFFT factory: Attempti
ng to register factory for plugin cuFFT when one has already been registered
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
E0000 00:00:1750626939.383181       1 cuda_dnn.cc:8579] Unable to register cuDNN factory: Attempting to register factory for plugin c
uDNN when one has already been registered
E0000 00:00:1750626939.457615       1 cuda_blas.cc:1407] Unable to register cuBLAS factory: Attempting to register factory for plugin
 cuBLAS when one has already been registered
W0000 00:00:1750626940.182714       1 computation_placer.cc:177] computation placer already registered. Please check linkage and avoi
d linking the same target more than once.
W0000 00:00:1750626940.182830       1 computation_placer.cc:177] computation placer already registered. Please check linkage and avoi
d linking the same target more than once.
W0000 00:00:1750626940.182838       1 computation_placer.cc:177] computation placer already registered. Please check linkage and avoi
d linking the same target more than once.
W0000 00:00:1750626940.182841       1 computation_placer.cc:177] computation placer already registered. Please check linkage and avoi
d linking the same target more than once.
2025-06-22 21:15:40.250639: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available
 CPU instructions in performance-critical operations.
To enable the following instructions: AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
2025-06-22 21:15:48.017091: E external/local_xla/xla/stream_executor/cuda/cuda_platform.cc:51] failed call to cuInit: INTERNAL: CUDA
error: Failed call to cuInit: UNKNOWN ERROR (303)
WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile_metrics` will be empty until yo
u train or evaluate the model.
INFO:     Started server process [1]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8080 (Press CTRL+C to quit)
```

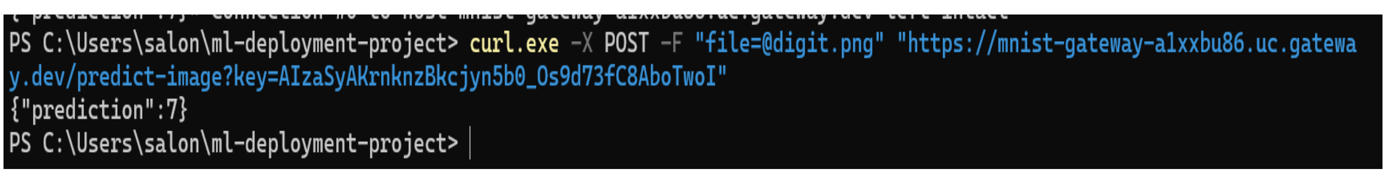
Fig 5. Running Docker Image Locally

Fig 6. Digit.png



```
atest PowerShell for new features and improvements! https://aka.ms/PSWindows
                                                                PS C:\Users\salon> curl http://localhos
t:8080/

StatusCode        : 200
StatusDescription : OK
Content           : {"message":"MNIST API is running!"}
RawContent        : HTTP/1.1 200 OK
                    Content-Length: 35
                    Content-Type: application/json
                    Date: Sun, 22 Jun 2025 21:15:57 GMT
                    Server: uvicorn

                    {"message":"MNIST API is running!"}
Forms             : {}
Headers           : {[Content-Length, 35], [Content-Type, application/json], [Date, Sun, 22 Jun 2025 21:15:57 GMT],
                    [Server, uvicorn]}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 35



PS C:\Users\salon> curl.exe -X POST -F "file=@digit.png" http://localhost:8080/predict-image
curl: (26) Failed to open/read local data from file/application
PS C:\Users\salon> cd .\ml-deployment-project
PS C:\Users\salon\ml-deployment-project> curl.exe -X POST -F "file=@digit.png" http://localhost:8080/predict-image
{"prediction":7}
PS C:\Users\salon\ml-deployment-project>
```

Fig 7. Predicted output (Locally)



```
PS C:\Users\salon\ml-deployment-project> curl.exe -X POST -F "file=@digit.png" "https://mnist-gateway-a1xxbu86.uc.gatewa
y.dev/predict-image?key=AIzaSyAKrnknzBkcjyn5b0_Os9d73fC8AboTwoI"
{"prediction":7}
PS C:\Users\salon\ml-deployment-project>
```

Fig 8. Predicted output (Using GCP)